

# REFERENCIA RÁPIDA DE PYTHON

FPI/FCyP



FACULTAD DE  
**INGENIERÍA**  
UNIVERSIDAD DE SANTIAGO DE CHILE

## Tipos Básicos

|                    |                   |                   |
|--------------------|-------------------|-------------------|
| <code>int</code>   | Números enteros   | 42                |
| <code>float</code> | Números flotantes | 1.618             |
| <code>bool</code>  | Valores lógicos   | <code>True</code> |
| <code>str</code>   | Strings           | "Monty"           |
| <code>list</code>  | Listas            | [1, 2, 3]         |

## Operaciones Básicas

|                               |                                                                         |
|-------------------------------|-------------------------------------------------------------------------|
| <code>input("Mensaje")</code> | Muestra <code>Mensaje</code> y recibe entrada por teclado del usuario   |
| <code>print(valor,...)</code> | Muestra por pantalla los valores separados por coma                     |
| <code>x = y</code>            | Asignación, <code>x</code> toma el valor de la expresión <code>y</code> |

## Operadores Aritméticos

A igual nivel de precedencia (P) en las operaciones, se agrupan de a pares de izquierda a derecha.

| Ejemplo             | Operación                    | P | T <sup>a</sup> |
|---------------------|------------------------------|---|----------------|
| <code>x ** y</code> | Exponenciación <sup>b</sup>  | 1 | B              |
| <code>+ x</code>    | Identidad                    | 2 | U              |
| <code>- x</code>    | Cambio de signo              | 2 | U              |
| <code>x * y</code>  | Multiplicación               | 3 | B              |
| <code>x / y</code>  | División                     | 3 | B              |
| <code>x // y</code> | División entera              | 3 | B              |
| <code>x % y</code>  | Módulo (resto <sup>c</sup> ) | 3 | B              |
| <code>x + y</code>  | Suma                         | 4 | B              |
| <code>x - y</code>  | Resta                        | 4 | B              |

Los operadores **unarios** (U) toman solo un operando.  
Los operadores **binarios** (B) toman dos.  
Todos los operadores aritméticos binarios tienen una versión de asignación como `+=`, que se usa como en el ejemplo:

`x += y` se interpreta como `x = x + y`

<sup>a</sup>Tipo

<sup>b</sup>Se agrupan de derecha a izquierda.

<sup>c</sup>De la división entera.

## Operadores lógicos

A igual nivel de precedencia en las operaciones, se agrupan de a pares de izquierda a derecha.

| Ejemplo                 | Operación         | P | T |
|-------------------------|-------------------|---|---|
| <code>x &gt; y</code>   | Mayor que         | 5 | B |
| <code>x &gt;= y</code>  | Mayor o igual que | 5 | B |
| <code>x &lt; y</code>   | Menor que         | 5 | B |
| <code>x &lt;= y</code>  | Menor o igual que | 5 | B |
| <code>x == y</code>     | Igual que         | 5 | B |
| <code>x != y</code>     | Distinto que      | 5 | B |
| <code>x in l</code>     | Pertenece a       | 5 | B |
| <code>x not in l</code> | No pertenece a    | 5 | B |
| <code>not p</code>      | Negación          | 6 | U |
| <code>p and q</code>    | Y lógico          | 7 | B |
| <code>p or q</code>     | O lógico          | 8 | B |

## Decisiones con if

Ejecutar sentencias que solo deben ocurrir al cumplirse la `<condición>` (**expresión booleana**):

```
if <condición>:  
    <sentencias condicionadas>
```

Ejecutar sentencias que solo deben ocurrir al cumplirse una condición, y otras en caso que no:

```
if <condición>:  
    <sentencias condicionadas>  
else:  
    <sentencias alternativas>
```

Ejecutar sentencias que solo deben ocurrir al cumplirse una condición, otras en caso de que no se cumpla la primera condición, pero si una segunda condición<sup>a</sup>, y otras en caso de que no se cumpla la primera ni la segunda:

```
if <condición 1>:  
    <sentencias condicionadas 1>  
elif <condición 2>:  
    <sentencias condicionadas 2>  
else:  
    <sentencias alternativas>
```

<sup>a</sup>Tantas condiciones secundarias (**elif**) como se necesite.

## Bloques de decisiones

Cada secuencia **if**, **if-else** o **if-elif-else** es un bloque independiente.

Dentro de un mismo bloque se ejecutarán las sentencias condicionadas a la primera condición válida comprobada.

Si existieran 2 bloques de decisión consecutivos se ejecutarán las sentencias condicionadas a la primera condición válida para cada bloque de manera independiente.

## Ciclos con while

Realizar una repetición condicionada de sentencias:

```
while <condición>:  
    <sentencias_a_repetir>
```

Donde `condición` es una **expresión booleana** y `<sentencias_a_repetir>` (última instrucción indentada con respecto a **while**) es la secuencia de instrucciones a repetir.

A través de este bloque, se asegura la ejecución de las `<sentencias_a_repetir>` mientras se cumpla la condición señalada.

Se puede repetir cualquier sentencia, incluyendo otros ciclos.

## Banderas (Flags)

Parte de la `<condición>` puede ser una variable booleana: esta indica si se debe continuar o no, según qué ocurra en el ciclo:

```
i = 1  
keep_going = True  
while keep_going and i <= 5:  
    if i % 2 == 0:  
        keep_going = False  
    i += 1
```

Haciendo uso de decisiones (**if**) y de una bandera (`keep_going`), se puede verificar si se cumple una condición adicional bajo la cual detener el ciclo antes de que el iterador `i` llegue a su límite.

## Listas

|                                 |                                                                                                                                      |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>lista = [1, 2, 3]</code>  | Define un objeto de tipo lista.                                                                                                      |
| <code>lista[i]</code>           | Retorna el elemento en la posición <i>i</i> . Soporta de 0 a $n-1^a$ de izquierda a derecha y de $-1$ a $-n$ de derecha a izquierda. |
| <code>lista.append(4)</code>    | Añade el elemento 4 al final de la lista.                                                                                            |
| <code>lista[j] = z</code>       | Redefine el valor del elemento en la posición <i>j</i> de la lista a <i>z</i> .                                                      |
| <code>lista.pop(k)</code>       | Retorna el elemento en la posición <i>k</i> y lo elimina de <i>lista</i> . Sin parámetros retorna y elimina el último elemento.      |
| <code>lista.count(c)</code>     | Retorna la cantidad de apariciones del elemento <i>c</i> .                                                                           |
| <code>lista.index(d)</code>     | Retorna la posición del elemento <i>d</i> .                                                                                          |
| <code>lista.remove(e)</code>    | Elimina la primera aparición del elemento <i>e</i> .                                                                                 |
| <code>lista.insert(i, f)</code> | Inserta el elemento <i>f</i> en la posición <i>i</i> <sup>b</sup> .                                                                  |
| <code>lista.sort()</code>       | Ordena <i>lista</i> en orden creciente.                                                                                              |
| <code>lista1 + lista2</code>    | Retorna una lista que concatena <i>lista1</i> y <i>lista2</i> .                                                                      |
| <code>lista * n</code>          | Retorna una lista que concatena <i>lista</i> <i>n</i> veces <sup>c</sup> .                                                           |

<sup>a</sup>Con *n* equivalente al largo de la lista.

<sup>b</sup>Los elementos siguientes son desplazados en 1 posición a la derecha.

<sup>c</sup>*n* debe ser entero.

## Largo de una Secuencia

El largo de la secuencia *seq* se obtiene mediante la función nativa `len(seq)`. Se entiende por “largo” de una secuencia como el total de elementos que hay en esta.

En el caso de una lista, si tiene como elemento otra lista, este elemento sigue contando como uno solo.

## Segmentos y Copias

Pueden accederse **segmentos** de una secuencia como una lista o string mediante la **notación slice**. Su notación básica es

`objeto[a:b:c]`,

retornando un objeto del mismo tipo del seccionado, con *a* el índice inicial del segmento, *b* el índice final (el último índice retornado será siempre el mayor valor posible menor a *b*) y *c* la distancia entre dos elementos consecutivos recuperados. *a* y *b* deben ser índices válidos y *c* debe ser un número entero (positivo o negativo).

Esta notación tiene las siguientes variaciones:

- Si se omite *a* (pero no *:*), inicia desde el comienzo.
- Si se omite *b* (pero no *:*), llega hasta el final.
- Si se omite *c* (incluyendo o no *:*), se asume 1.

## Consultas sobre Strings

Todos estos métodos devuelven **True** o **False**.

|                              |                                                     |
|------------------------------|-----------------------------------------------------|
| <code>s.isupper()</code>     | Todas las letras son mayúsculas y hay al menos una. |
| <code>s.islower()</code>     | Todas las letras son minúsculas y hay al menos una. |
| <code>s.isalpha()</code>     | Todos los caracteres son alfabéticos.               |
| <code>s.isdigit()</code>     | Todos los caracteres son dígitos.                   |
| <code>s.isalnum()</code>     | Todos los caracteres son alfabéticos o dígitos.     |
| <code>s.endswith(t)</code>   | El string termina con <i>t</i> .                    |
| <code>s.startswith(t)</code> | El string empieza con <i>t</i> .                    |

## String a Lista y Viceversa

- `s.split(sep)` separa el string utilizando *sep* como separador y retorna una lista cuyos elementos son los fragmentos del string.
- `sep.join(lista)` une los elementos de *lista*, separados por *sep*, en un único string. Solo funciona si los elementos son strings.

## Strings

|                             |                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>s = "Monty"</code>    | Define un objeto de tipo string.                                                                          |
| <code>s = 'Python'</code>   | Retorna el índice donde empieza <i>sub</i> .                                                              |
| <code>s.find(sub)</code>    | Como <i>find</i> , pero desde la derecha.                                                                 |
| <code>s.rfind(sub)</code>   | Como <i>find</i> , pero arroja error si no encuentra.                                                     |
| <code>s.index(sub)</code>   | Retorna el string en minúscula.                                                                           |
| <code>s.lower()</code>      | Retorna el string en mayúscula.                                                                           |
| <code>s.upper()</code>      | Retorna el string eliminando los caracteres en <i>t</i> de los extremos del string.                       |
| <code>s.strip(t)</code>     | Como <i>strip</i> , pero eliminando espacios en blanco.                                                   |
| <code>s.strip()</code>      | Retorna el string convertido con el primer caracter a mayúscula, si es una letra, y el resto a minúscula. |
| <code>s.capitalize()</code> | Retorna el string en “Formato De Título”.                                                                 |
| <code>s.title()</code>      | Retorna la cantidad de apariciones no superpuestas de <i>sub</i> .                                        |
| <code>s.count(sub)</code>   |                                                                                                           |

## Funciones Nativas

Estas funciones, entre otras, están siempre disponibles:

|                               |                                                    |
|-------------------------------|----------------------------------------------------|
| <code>abs(x)</code>           | Valor absoluto de <i>x</i> .                       |
| <code>max(seq)</code>         | Retorna el máximo de <i>seq</i> .                  |
| <code>min(seq)</code>         | Retorna el mínimo de <i>seq</i> .                  |
| <code>range(stop)</code>      | Genera una secuencia de valores enteros.           |
| <code>range(i, f, s)</code>   | Redondea <i>x</i> a entero o a <i>n</i> decimales. |
| <code>round(x)</code>         | Retorna la secuencia <i>seq</i> ordenada           |
| <code>round(x, n)</code>      | Retorna el tipo de <i>o</i>                        |
| <code>sorted(seq)</code>      | Verifica si <i>o</i> es del tipo <i>c</i>          |
| <code>type(o)</code>          |                                                    |
| <code>isinstance(o, c)</code> |                                                    |

El nombre de cada tipo de dato es la función para cambiar a dicho tipo de dato, p.e.: `x = int(y)` o `l = list(s)` (si *s* es un conjunto o secuencia, como un string).

## Funciones Importadas

Un módulo es un archivo que contiene definiciones y declaraciones en Python.

Existen módulos estándar (siempre disponibles para importar), definidos por el programador y externos (instalados en el sistema).

Para importar, utilizamos:

```
import module
# Uso de, por ejemplo, func1:
module.func1(params)
# Alternativamente, podemos dar un
# alias
import module as bla
bla.func1(params)
```

En casos especiales, podemos importar solo las funciones de interés utilizando:

```
from module import func1, func2, func3
```

De modo que se pueda utilizar `func1`, `func2` y `func3` como si fuesen nativas.

## Definición de Funciones Propias

La estructura básica de definición de funciones es:

```
def func_name(arg1, arg2, arg3=valor):
    definiciones_y_operaciones_locales
    return resultado
```

- Dondequiera que se encuentre `return`, se finaliza la función y se entrega ese resultado. Puede haber varios, según si los resultados son condicionados.
- Los parámetros son tantos como se requieran. Si no se requieren, los paréntesis van vacíos.
- Los parámetros opcionales se escriben al final, dándoles su valor por defecto (como en `arg3`).

Una vez definida, se llama como una función nativa normal:

```
# Solo parámetros obligatorios
r = func_name(x, y)
# Con el parámetro opcional
r = func_name(x, y, z)
# Dando nombre del parámetro opcional
r = func_name(x, y, arg3=z)
```

## Comentarios y Documentación

Las líneas comenzadas por `#` son comentarios e ignorados por el intérprete. Sirven para documentar el código para quien lo lea.

Las funciones se pueden documentar con *docstrings*, explicando su utilidad y forma de uso inmediatamente después de definirlas:

```
def ejemplo(arg1, arg2=valor):
    """Descripción de lo que hace la
    función

    Entradas: Se describen sus entradas
    (incluyendo tipo de dato)
    Salidas: Se describen sus salidas
    (incluyendo tipo de dato)
    """
    operaciones
    return resultado
```

Las docstrings son un tipo de string multilínea, pueden escribirse con triple comilla simple (`'''así'''`) o triple comilla doble (`"""así"""`).

**N.B.:** Los comentarios son para quien leerá o modificará el código, las docstrings son para quien utilizará la función en el suyo propio.

## Módulos Estándar

|                       |                                              |
|-----------------------|----------------------------------------------|
| <code>math</code>     | Funciones matemáticas.                       |
| <code>sys</code>      | Funciones y parámetros del intérprete.       |
| <code>op.path</code>  | Manipulaciones comunes de rutas de archivo.  |
| <code>random</code>   | Generación de números pseudo aleatorios.     |
| <code>datetime</code> | Tipos básicos de fecha y hora.               |
| <code>copy</code>     | Operaciones de copia superficial y profunda. |
| <code>time</code>     | Operaciones de tiempo.                       |

## Importación de Módulos Propios

Si escribimos funciones y constantes en un archivo independiente, puede importarse como cualquier otro módulo:

```
# Si utilidades.py existe en la
# misma carpeta
import utilidades
from utilidades import func
```

Se pueden importar nombres de funciones y constantes.

El nombre del módulo no puede contener espacios.

## Recursión

Python solo requiere el nombre de la función para poder utilizarla, por lo que podemos construir una función en base a sí misma:

```
def func(x):
    if condición(x):
        new_x = operaciones(x)
        return func(new_x)
    else:
        return otras_operaciones(x)
```